

Содержание

1. Что это за лабораторная	3
2. Структура папки <code>lab4</code>	3
3. Теория: почему для изображений используют CNN	4
3.1. Почему обычный MLP плохо подходит для изображений	4
3.2. Свёртка	4
3.3. Padding, kernel size, stride	5
3.4. Нелинейность ReLU	5
3.5. Pooling	6
3.6. Почему CNN хорошо работает иерархически	6
3.7. Классификация и CrossEntropyLoss	6
3.8. Accuracy, validation и test	7
3.9. Early stopping	7
4. Как устроены данные в этой работе	7
4.1. Что такое MNIST и FashionMNIST	7
4.2. Нормализация	8
4.3. Подвыборки и ускорение экспериментов	8
4.4. Разделение train и val	8
4.5. DataLoader	9
5. Разбор <code>config.py</code> : что именно настраивается	9
5.1. Датасет	9
5.2. Размеры выборок	9
5.3. Обучение	10
5.4. Предобработка и DataLoader	10
5.5. Демонстрация переобучения	10
6. Разбор <code>snn.py</code> : сердце лабораторной	10
6.1. Выбор устройства	10
6.2. Фиксация случайности	10
6.3. Два типа свёрточных блоков	11
6.3.1. ConvBlockA	11
6.3.2. ConvBlockB	11
6.4. Класс ImageCNN	11
6.5. Проверка допустимости конфигурации	12
6.6. Оптимизаторы	12
6.7. <code>train_one_epoch(...)</code>	12
6.8. <code>evaluate(...)</code>	13
6.9. <code>train_model(...)</code>	13
7. Блок 1: перебор гиперпараметров CNN	13
7.1. Сетка конфигураций	13
7.2. Почему конфигурации нумеруются	14

7.3. Запуск одного эксперимента	14
7.4. Общий сценарий <code>main()</code>	14
7.5. Как выбирается лучшая модель	15
7.6. Финальное обучение на полном <code>train</code>	15
7.7. Что показывают графики в блоке 1	16
7.7.1. <code>plot_grid_summary(results)</code>	16
7.7.2. <code>plot_curves(history, ...)</code>	16
7.7.3. <code>plot_confusion(...)</code>	16
7.7.4. <code>plot_param_compare(...)</code>	16
7.8. Как интерпретировать результаты блока 1	17
8. Блок 2: оптимизаторы и переобучение	17
8.1. Базовая архитектура	17
8.2. Сравнение оптимизаторов	17
8.3. Что здесь нужно увидеть по смыслу	18
8.4. Эксперимент на переобучение	18
8.5. Роль Dropout	18
8.6. Функция <code>plot_overfit_compare(...)</code>	19
9. Как читать результаты обучения правильно	19
9.1. Нормальное обучение	19
9.2. Недообучение	19
9.3. Переобучение	20
9.4. Быстрая, но нестабильная оптимизация	20
10. Почему эта реализация хороша с инженерной точки зрения	20
10.1. Разделение ответственности по файлам	20
10.2. Повторное использование функций	20
10.3. Воспроизводимость	21
10.4. Корректный ML-пайплайн	21
11. Какой путь сигнала проходит один объект в сети	21
12. Что стоит понимать особенно хорошо для защиты или отчёта	21
13. Что можно улучшить в дальнейшем	22
14. Краткий итог по лабораторной	22
15. Практический порядок чтения кода после этого гайда	23

1. Что это за лабораторная

Лабораторная работа 4 посвящена **свёрточным нейронным сетям** (CNN, Convolutional Neural Networks) для классификации изображений. В этой реализации используются датасеты **MNIST** и **FashionMNIST**, а сама работа разбита на два смысловых блока:

- **Блок 1** — перебор архитектур и гиперпараметров CNN, выбор лучшей конфигурации по `val accuracy`, финальное дообучение на полном `train` и оценка на `test`.
- **Блок 2** — дополнительные исследования: сравнение оптимизаторов (`Adam`, `AdamW`, `SGD`) и демонстрация переобучения с регуляризацией и без неё.

Если формулировать совсем кратко, то эта лабораторная отвечает на три главных вопроса:

- как устроена CNN и почему она лучше обычного MLP для изображений;
- как организовать корректный эксперимент по подбору архитектуры;
- как по графикам и метрикам понять, что модель обучается хорошо, переобучается или обучается слишком медленно.

2. Структура папки `lab4`

Практически вся логика лабораторной вынесена в несколько небольших файлов, и это хорошо: каждый модуль отвечает за свою часть эксперимента.

Файл	Назначение
<code>config.py</code>	Все основные настройки эксперимента: датасет, размеры выборок, число эпох, <code>early stopping</code> , параметры <code>DataLoader</code> и демонстрации переобучения.
<code>data.py</code>	Загрузка MNIST / FashionMNIST , нормализация, взятие подвыборок, разбиение <code>train</code> -> <code>train</code> + <code>val</code> , создание <code>DataLoader</code> , визуализация случайных изображений.
<code>cnn.py</code>	Архитектура модели, свёрточные блоки, фабрика оптимизаторов, вычисление метрик, функции <code>train_one_epoch</code> , <code>evaluate</code> , <code>train_model</code> .
<code>block1_main.py</code>	Основной сценарий перебора конфигураций CNN, выбора лучшей архитектуры, финального обучения и построения графиков.
<code>block2_main.py</code>	Сравнение оптимизаторов и отдельный сценарий показа переобучения и влияния <code>Dropout</code> .
<code>4_CNN.ipynb</code>	Исходный notebook, чью логику Python-скрипты воспроизводят в более структурированном виде.

Хороший способ читать эту лабу — идти в таком порядке:

1. Сначала понять теорию CNN.

2. Потом прочитать `config.py`, чтобы увидеть, что именно настраивается.
3. Затем `data.py`, чтобы понять, как формируются данные.
4. После этого `cnn.py`, потому что там находится сама модель и цикл обучения.
5. И только затем уже `block1_main.py` и `block2_main.py`, которые собирают всё вместе в эксперименты.

3. Теория: почему для изображений используют CNN

3.1. Почему обычный MLP плохо подходит для изображений

Если подать изображение размера 28×28 в обычный многослойный перцептрон, то его обычно приходится сначала развернуть в вектор длины 784. После этого информация о том, какие пиксели были соседями по горизонтали и вертикали, почти теряется на уровне архитектуры: сеть видит просто длинный список чисел.

Для изображений это неудобно, потому что нам важны **локальные шаблоны**:

- края;
- линии;
- углы;
- маленькие фрагменты формы;
- композиция таких фрагментов в более сложные объекты.

CNN создавалась именно под такую структуру данных. Она использует две ключевые идеи:

- **локальные receptive fields** — нейрон смотрит не на всё изображение сразу, а на небольшой фрагмент;
- **shared weights** — один и тот же фильтр применяется ко всем участкам изображения, то есть сеть учится искать один и тот же тип признака в разных местах.

За счёт этого CNN:

- лучше учитывает геометрию изображения;
- требует меньше параметров, чем полносвязная сеть схожей мощности;
- обычно обучается устойчивее на задачах компьютерного зрения.

3.2. Свёртка

Свёрточный слой `Conv2d` применяет к входному изображению набор обучаемых фильтров. Каждый фильтр — это маленькое ядро, например 3×3 или 5×5 , которое скользит по изображению и вычисляет отклик.

Если обозначить входную карту признаков как x , ядро как K , то одна свёртка по смыслу выглядит так:

$$y(i, j) = \sum_u \sum_v x(i + u, j + v) K(u, v) + b.$$

Интерпретация очень важна:

- ядро не просто усредняет пиксели;
- его коэффициенты обучаются градиентным спуском;
- поэтому один фильтр может научиться реагировать на вертикальные границы, другой — на дуги, третий — на штрихи, четвёртый — на текстуру ткани и так далее.

Если фильтров несколько, то на выходе слоя получается не одна карта признаков, а целый набор каналов. Именно поэтому в коде есть параметр `channels`, например (16, 32) или (32, 64): это число выходных каналов по блокам.

3.3. Padding, kernel size, stride

В лабораторной регулируются важные параметры свёртки:

- `conv_k` — размер ядра свёртки;
- `conv_s` — шаг свёртки;
- `pad = conv_k // 2` — padding, который подбирается так, чтобы при `stride = 1` примерно сохранять пространственный размер;
- `pool_k`, `pool_s` — размер и шаг pooling.

Для двумерной свёртки выходной размер по одной координате вычисляется формулой

$$\text{out} = \left\lfloor \frac{\text{in} + 2p - k}{s} + 1 \right\rfloor,$$

где:

- `in` — входной размер;
- `p` — padding;
- `k` — размер ядра;
- `s` — stride.

Именно по этой логике в функции `is_valid_config(...)` из `cnn.py` проверяется, не схлопнется ли изображение в ноль после нескольких блоков подряд.

3.4. Нелинейность ReLU

После свёртки в этой работе везде используется ReLU:

$$\text{ReLU}(z) = \max(0, z).$$

Её смысл простой:

- положительные отклики пропускаются дальше;
- отрицательные зануляются;

- сеть получает нелинейность, без которой несколько линейных операций подряд были бы эквивалентны одной линейной операции.

Практически ReLU хорошо работает в CNN, вычислительно дешёва и обычно ускоряет обучение.

3.5. Pooling

После свёртки в блоках используется `MaxPool2d`. Эта операция уменьшает пространственный размер карт признаков и оставляет наиболее сильные отклики в локальных окнах.

Интуитивно max-pooling делает сразу несколько полезных вещей:

- уменьшает размер тензора;
- снижает вычислительную нагрузку на последующие слои;
- даёт некоторую устойчивость к небольшим сдвигам объекта на изображении;
- помогает выделять сильные локальные признаки.

3.6. Почему CNN хорошо работает иерархически

Глубокая CNN обычно учит признаки по уровням:

- первые слои учат простые контуры и переходы яркости;
- средние слои собирают из них более сложные шаблоны;
- глубокие слои распознают уже почти целые части объекта или характерные композиции признаков.

Для MNIST это можно понимать так:

- на нижних слоях сеть видит штрихи и края цифры;
- выше — дуги, пересечения, вертикальные и горизонтальные сегменты;
- ещё выше — уже характерную форму 3, 5, 8 и так далее.

3.7. Классификация и CrossEntropyLoss

На выходе модель должна выбрать один из 10 классов. Линейный слой выдаёт вектор **логитов** $z \in \mathbb{R}^{10}$. Затем функция потерь `CrossEntropyLoss` внутри объединяет два шага:

- `log_softmax`;
- отрицательное логарифмирование вероятности правильного класса.

Если истинный класс объекта равен y , то loss для одного примера можно понимать как

$$L = -\log p_y,$$

где

$$p_c = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

Следствие очень важное: модель обучается не просто выдавать правильный индекс класса, а повышать уверенность именно правильного класса относительно остальных.

3.8. Accuracy, validation и test

В лабораторной используются три выборки:

- **train** — по ней считаются градиенты и обновляются веса;
- **val** — по ней выбирают архитектуру и следят за качеством вне обучения;
- **test** — используется только для финальной честной оценки после выбора модели.

Это принципиально важно. Если выбирать лучшую модель сразу по **test**, то оценка становится смещённой: модель фактически подбирается под тест, а не проверяется на нём честно.

3.9. Early stopping

В функции `train_model(...)` реализована ранняя остановка по `val_acc`.

Идея такая:

- если `val accuracy` заметно улучшается, сохраняем текущее состояние модели;
- если улучшения нет несколько эпох подряд, обучение останавливаем;
- после остановки загружается лучшее сохранённое состояние.

Это защищает от двух проблем:

- зря не тратить эпохи после выхода на плато;
- не уходить слишком далеко в переобучение.

Параметры ранней остановки в этой лабораторной:

- `PATIENCE` — сколько эпох можно терпеть без улучшения;
- `MIN_DELTA` — какое улучшение считать значимым.

4. Как устроены данные в этой работе

4.1. Что такое MNIST и FashionMNIST

MNIST — классический датасет рукописных цифр. Каждый объект — это серое изображение размера 28×28 и метка класса от 0 до 9.

FashionMNIST устроен так же по формату, но вместо цифр содержит 10 классов предметов одежды и обуви. Он обычно сложнее, чем обычный **MNIST**, потому что классы визуально ближе друг к другу.

В `data.py` это отражено через словарь `DATASET_INFO`, где для каждого датасета задаются:

- класс загрузчика `torchvision.datasets`;
- список имён классов;
- число каналов `in_channels`;
- размер изображения `image_size`.

4.2. Нормализация

В `data.py` есть словарь `DATASET_NORMALIZATION`, а при загрузке в `load_image_dataset(...)` последовательно строятся преобразования:

```
tfms = [transforms.ToTensor()]
if NORMALIZE_IMAGES:
    tfms.append(transforms.Normalize(mean, std))
```

Зачем нормализация нужна:

- стабилизирует распределение входов;
- облегчает оптимизацию;
- ускоряет и делает более устойчивым обучение.

Формально каждый пиксель преобразуется так:

$$x' = \frac{x - \mu}{\sigma}.$$

4.3. Подвыборки и ускорение экспериментов

В `config.py` есть параметры:

- `TRAIN_SUBSET`;
- `TEST_SUBSET`;
- `VAL_RATIO`.

Это сделано очень разумно: во время перебора архитектур не обязательно каждый раз гонять все 60 тысяч изображений MNIST. Для сетки гиперпараметров можно взять уменьшенную выборку, быстро сравнить конфигурации, а уже после выбора лучшей модели выполнить финальное обучение на полном `train`.

Такой дизайн даёт хороший баланс между скоростью и корректностью эксперимента.

4.4. Разделение `train` и `val`

Функция `split_train_val(dataset, val_ratio, seed=SEED)` делает случайное разбиение обучающей части на:

- новый `train_ds`;

- `val_ds`.

С технической точки зрения:

- создаётся массив индексов;
- индексы случайно перемешиваются;
- первые индексы идут в `train`, последние — в `val`.

Важно, что везде используется фиксированный `seed`. Это делает эксперимент воспроизводимым: при одинаковых параметрах мы получим тот же набор индексов и сопоставимые результаты.

4.5. DataLoader

Функция `make_loaders(...)` превращает наборы данных в `DataLoader`, которые отдают батчи.

Ключевые параметры:

- `batch_size` — размер батча;
- `shuffle=True` для `train`;
- `shuffle=False` для `val/test`;
- `num_workers` — число параллельных процессов загрузки;
- `pin_memory` — ускорение передачи на GPU, если оно доступно.

Практический смысл такой:

- `train` нужно перемешивать, чтобы обучение было менее чувствительным к порядку примеров;
- `val` и `test` не нужно перемешивать, потому что там мы только оцениваем модель.

5. Разбор `config.py`: что именно настраивается

Этот файл задаёт всю конфигурацию эксперимента через переменные окружения `SMGM04_*`. Это очень удобный подход: можно менять поведение запуска, не редактируя код.

Логические группы параметров:

5.1. Датасет

- `DATASET_NAME` — MNIST или FashionMNIST.
- `DATA_ROOT` — путь к папке с данными.

5.2. Размеры выборок

- `TRAIN_SUBSET` — сколько объектов брать для перебора конфигураций.
- `TEST_SUBSET` — сколько объектов брать для промежуточной тестовой оценки.
- `VAL_RATIO` — доля validation внутри train-подвыборки.

5.3. Обучение

- `BATCH_SIZE` — размер батча.
- `N_EPOCHS` — максимум эпох в основном обучении.
- `PATIENCE` — patience для early stopping.
- `MIN_DELTA` — минимальный прирост `val_acc`, который считается улучшением.
- `TARGET_ACC` — целевой accuracy, например `0.90`.
- `FINAL_EPOCHS` — число эпох для финального дообучения лучшей модели.

5.4. Предобработка и DataLoader

- `NORMALIZE_IMAGES` — включать ли нормализацию.
- `NUM_WORKERS` — число воркеров загрузки.
- `PIN_MEMORY` — автоматический режим или явное включение/выключение.

5.5. Демонстрация переобучения

- `OVERFIT_TRAIN` — размер маленькой train-выборки для эксперимента на переобучение.
- `OVERFIT_EPOCHS` — сколько эпох крутить этот эксперимент.

Главная идея `config.py`: сделать лабораторную не жёстко зашитой, а экспериментальной. Это хороший инженерный ход, потому что одна и та же логика легко запускается в быстрых и в более тщательных режимах.

6. Разбор `snp.py`: сердце лабораторной

6.1. Выбор устройства

Функция `get_device()` выбирает устройство в таком порядке:

- `cuda`, если доступна NVIDIA GPU;
- `mps`, если доступен Apple Metal;
- иначе `cpu`.

Это особенно актуально на macOS, где `mps` позволяет использовать GPU Apple Silicon.

6.2. Фиксация случайности

Функция `set_seed(...)` устанавливает seed для:

- стандартного `random`;
- `numpy`;
- `torch`;
- при наличии `cuda` — и для всех CUDA-устройств.

Это нужно для воспроизводимости. Без фиксированного seed сравнение архитектур было бы менее честным: часть различий могла бы объясняться не архитектурой, а случайностью инициализации и разбиения данных.

6.3. Два типа свёрточных блоков

В коде предусмотрены два варианта блока.

6.3.1. ConvBlockA

```
Conv2d -> ReLU -> MaxPool2d
```

Это более простой блок. Он:

- легче по числу параметров;
- быстрее обучается;
- может хуже извлекать сложные признаки, чем более глубокий блок.

6.3.2. ConvBlockB

```
Conv2d -> ReLU -> Conv2d -> ReLU -> MaxPool2d
```

Это более мощный блок, потому что до pooling выполняются две свёртки подряд. Это позволяет:

- извлечь более сложные локальные признаки;
- увеличить выразительную способность блока;
- но цена — больше параметров и вычислений.

Очень важно заметить: `block_type='b'` не просто добавляет один слой, а меняет характер обработки признаков внутри каждого блока.

6.4. Класс ImageCNN

Это основной класс модели. Его идея:

1. Выбрать тип блока A или B.
2. Построить последовательность таких блоков по списку `channels`.
3. После свёрточной части выполнить глобальное усреднение `AdaptiveAvgPool2d((1, 1))`.
4. Превратить результат в вектор через `Flatten()`.
5. Применить `Dropout`.
6. Классифицировать через финальный `Linear(prev, num_classes)`.

Почему `AdaptiveAvgPool2d((1, 1))` — хорошее решение:

- не нужно вручную высчитывать размер тензора перед полносвязным слоем;
- каждую карту признаков мы сводим к одному среднему значению;

- модель становится проще и более универсальной по форме входного тензора.

После этой операции размерность тензора становится примерно такой:

$$(\text{batch}, \text{channels}, 1, 1) \rightarrow (\text{batch}, \text{channels}).$$

То есть каждый выходной канал интерпретируется как один высокоуровневый признак.

6.5. Проверка допустимости конфигурации

Функция `is_valid_config(...)` проверяет, что после последовательности свёрток и pooling пространственный размер не станет меньше 1.

Это хороший пример инженерной защиты от некорректной архитектуры. Если такую проверку не делать, часть конфигураций из сетки могла бы падать уже во время прямого прохода модели.

6.6. Оптимизаторы

Функция `make_optimizer(name, params, lr)` поддерживает:

- Adam;
- AdamW;
- SGD с momentum и `nesterov=True`.

Кратко по смыслу:

- Adam — обычно хороший стартовый выбор, быстро сходится;
- AdamW — похож на Adam, но аккуратнее работает с weight decay;
- SGD — может сходиться медленнее, но иногда даёт более хорошие обобщающие свойства при хорошей настройке learning rate.

6.7. `train_one_epoch(...)`

Это базовый цикл одной эпохи обучения:

1. `model.train()` включает режим обучения.
2. Для каждого батча данные отправляются на устройство.
3. Обнуляются градиенты `optimizer.zero_grad()`.
4. Выполняется прямой проход `out = model(x)`.
5. Считается loss.
6. Вызывается `loss.backward()`.
7. Выполняется шаг оптимизатора `optimizer.step()`.
8. Накапливаются суммарные loss и accuracy.

По сути это и есть классический mini-batch gradient descent.

6.8. `evaluate(...)`

Функция оценки работает под декоратором `@torch.no_grad()`, то есть градиенты не вычисляются. Это важно потому что:

- экономится память;
- ускоряются вычисления;
- режим оценки становится корректным с точки зрения inference.

Если `return_preds=True`, функция дополнительно возвращает:

- истинные метки;
- предсказанные метки.

Это нужно для построения confusion matrix.

6.9. `train_model(...)`

Это уже полный обучающий сценарий на много эпох с сохранением истории:

- `train_loss, val_loss`;
- `train_acc, val_acc`.

Логика функции:

1. Инициализируется лучшее значение `best_val_acc`.
2. Копируется стартовое состояние модели `best_state`.
3. На каждой эпохе вызываются `train_one_epoch(...)` и `evaluate(...)`.
4. История записывается в словарь `history`.
5. Если `val_acc` улучшилась на `MIN_DELTA`, сохраняется новое лучшее состояние.
6. Если улучшения нет, счётчик `bad` увеличивается.
7. При `bad >= patience` обучение прерывается.
8. После цикла загружается лучшее сохранённое состояние.

Это очень важная деталь: в конце возвращается не состояние последней эпохи, а лучшее по validation.

7. Блок 1: перебор гиперпараметров CNN

Теперь разберём главный эксперимент в `block1_main.py`.

7.1. Сетка конфигураций

В начале файла задаются варианты параметров:

```
BLOCK_TYPES = ['a', 'b']
CHANNELS_OPTIONS = [(16, 32), (32, 64)]
CONV_KERNELS = [3, 5]
CONV_STRIDES = [1]
```

```
POOL_KERNELS = [2]
POOL_STRIDES = [1, 2]
LEARNING_RATES = [1e-3]
```

Функция `build_grid(image_size=28)` перебирает декартово произведение этих списков и оставляет только валидные конфигурации через `is_valid_config(...)`.

Что именно здесь исследуется:

- глубина блока: одна свёртка внутри блока или две;
- ширина модели: (16, 32) против (32, 64);
- размер свёрточного ядра: 3 против 5;
- шаг pooling: 1 или 2.

Это уже полноценный гиперпараметрический поиск, пусть и по небольшой сетке.

7.2. Почему конфигурации нумеруются

Каждая конфигурация получает `id`. Это нужно не только для красоты вывода, но и для отслеживания экспериментов на графиках и в таблицах. Когда конфигураций много, краткая метка вида `#05 blk=32 ch=64 sk=3 ps=2` удобнее длинного текстового описания.

7.3. Запуск одного эксперимента

Функция `run_one(cfg, train_loader, val_loader, info, device)` делает всё необходимое для одной конфигурации:

1. Фиксирует `seed`.
2. Создает `ImageCNN` с параметрами из `cfg`.
3. Создает `Adam` с нужным `lr`.
4. Выбирает `CrossEntropyLoss`.
5. Считает число параметров.
6. Запускает `train_model(...)`.
7. Сохраняет в результат сам конфиг, модель и число параметров.

Заметь важный момент: **каждая конфигурация обучается с нуля**. Это обязательно для честного сравнения.

7.4. Общий сценарий `main()`

Функция `main()` из первого блока работает так:

1. Фиксирует `seed` и выбирает устройство.
2. Загружает `train_full`, `test_full`, `info`.
3. Берёт `train`-подвыборку `TRAIN_SUBSET` для сетки экспериментов.
4. Делит её на `train_ds` и `val_ds`.

5. Берёт test-подвыборку `TEST_SUBSET`.
6. Создаёт `DataLoader`.
7. Показывает случайные изображения из `train`.
8. Строит сетку конфигураций.
9. Для каждой конфигурации запускает `run_one(...)`.
10. Печатает сводную таблицу результатов.
11. Выбирает лучшую конфигурацию по `best_val_acc`.
12. Оценивает лучшую модель на test-подвыборке.
13. Переобучает лучший конфиг на полном `train`.
14. Считает финальные метрики и строит графики.

Этот сценарий хорошо соответствует правильной исследовательской логике машинного обучения.

7.5. Как выбирается лучшая модель

Лучшая конфигурация выбирается выражением по смыслу вида:

```
best = max(results, key=lambda result: result['best_val_acc'])
```

То есть критерий выбора — лучшая `validation accuracy`, а не `train accuracy` и не `test accuracy`.

Это правильно, потому что:

- `train accuracy` легко зависит переобучением;
- `test accuracy` нельзя использовать для подбора модели;
- `val accuracy` специально нужна именно для выбора архитектуры и гиперпараметров.

7.6. Финальное обучение на полном `train`

Функция `train_final_on_full(...)` делает отдельный, очень важный этап:

- берётся лучший конфиг;
- создаётся новая модель с нуля;
- она обучается уже не на `train-split` для сетки, а на полном `train_full_ds`;
- затем оценивается на `test_ds`.

Почему это разумно:

- модель уже выбрана;
- теперь можно использовать все доступные обучающие данные;
- это обычно улучшает финальное качество по сравнению с обучением только на `train-части` после `split`.

7.7. Что показывают графики в блоке 1

В конце первого блока строятся несколько видов визуализаций.

7.7.1. `plot_grid_summary(results)`

Показывает:

- `best val_acc` для каждой конфигурации;
- эпоху достижения целевого `TARGET_ACC`.

По этому графику удобно видеть:

- какие конфигурации вообще жизнеспособны;
- какие достигают цели быстрее;
- есть ли заметный разрыв между лидерами и остальными.

7.7.2. `plot_curves(history, ...)`

Показывает пары графиков:

- `train_loss` и `val_loss`;
- `train_acc` и `val_acc`.

Именно по этим кривым обычно диагностируют:

- нормальное обучение;
- недообучение;
- переобучение;
- плато.

7.7.3. `plot_confusion(...)`

Матрица ошибок показывает, какие классы сеть путает чаще всего. Это особенно полезно на `FashionMNIST`, где некоторые классы визуально похожи.

Например, если сеть часто путает `Shirt` и `T-shirt`, это уже содержательный вывод о природе задачи, а не просто число accuracy.

7.7.4. `plot_param_compare(...)`

Для каждого отдельного гиперпараметра строится агрегированное сравнение среднего `val_acc`.

Это помогает ответить на вопросы вида:

- полезнее ли блок `b`, чем блок `a`;
- действительно ли `(32, 64)` лучше, чем `(16, 32)`;
- влияет ли ядро `5x5` лучше или хуже, чем `3x3`;
- какой `pool stride` чаще выигрывает.

7.8. Как интерпретировать результаты блока 1

После выполнения блока 1 имеет смысл отвечать не только на вопрос «какая модель лучшая», но и на более глубокие вопросы:

1. **Даёт ли увеличение числа каналов прирост качества?** Если да, то насколько он стоит увеличения параметров и времени.
2. **Лучше ли глубокий блок **b**, чем **a**?** Если выигрыш небольшой, возможно, более простая модель выгоднее.
3. **Как влияет размер ядра?** Большое ядро видит больший контекст, но и параметров у него больше.
4. **Сколько эпох нужно до достижения целевого качества?** Это показатель скорости сходимости.
5. **Есть ли признаки переобучения?** Например, train accuracy растёт, а val accuracy стагнирует.

Правильный вывод по лабораторной должен быть не только численным, но и причинным: **почему** та или иная конфигурация выиграла.

8. Блок 2: оптимизаторы и переобучение

Второй блок не подбирает архитектуру, а исследует поведение уже выбранной хорошей архитектуры.

8.1. Базовая архитектура

В `block2_main.py` зафиксирован словарь:

```
BASE_CFG = {
    'block_type': 'b',
    'channels': (32, 64),
    'conv_k': 3, 'conv_s': 1,
    'pool_k': 2, 'pool_s': 2,
}
```

Смысл такой: сначала в блоке 1 мы ищем хорошую архитектуру, а потом уже на основе сильной архитектуры сравниваем влияние способа оптимизации и регуляризации.

8.2. Сравнение оптимизаторов

В списке `OPT_VARIANTS` задаются пары:

- ('Adam', 1e-3)
- ('Adam', 3e-3)
- ('AdamW', 1e-3)
- ('SGD', 1e-2)

- ('SGD', 5e-2)

Функция `study_optimizers(...)` для каждой пары:

1. фиксирует seed;
2. создаёт одинаковую архитектуру;
3. создаёт оптимизатор с конкретным lr;
4. обучает модель;
5. сохраняет кривые и лучшие метрики.

Это уже почти чистый controlled experiment: архитектура та же, данные те же, меняется в основном только алгоритм оптимизации.

8.3. Что здесь нужно увидеть по смыслу

Если Adam быстрее достигает высокой `val_acc`, это означает, что адаптивный подбор шага хорошо подходит для этой задачи.

Если SGD отстаёт в первых эпохах, это нормально: ему часто нужно больше эпох и более аккуратная настройка lr.

Если AdamW даёт чуть более устойчивое качество, это можно связать с более корректной регуляризацией через decoupled weight decay.

Главная мысль: **оптимизатор влияет не только на финальное качество, но и на траекторию обучения.**

8.4. Эксперимент на переобучение

Функция `run_overfit_demo(...)` делает очень полезный учебный эксперимент:

- берётся маленькая train-подвыборка `OVERFIT_TRAIN`;
- модель обучается долго;
- сравниваются два случая: без регуляризации и с `Dropout=0.4`.

Почему это важно:

- на маленьком train наборе модель легко запоминает данные;
- тогда train accuracy становится очень высокой;
- а validation accuracy растёт слабее или даже начинает деградировать;
- разрыв между train и val — классический признак переобучения.

8.5. Роль Dropout

В ImageCNN перед финальным линейным слоем стоит `Dropout(dropout_rate)`. Во втором блоке сравниваются:

- `dropout=0.0`;
- `dropout=0.4`.

Механика dropout такая:

- во время обучения часть признаков случайно зануляется;
- сеть не может слишком сильно опереться на один узкий набор признаков;
- это снижает коадаптацию нейронов и часто улучшает обобщение.

На уровне наблюдаемых графиков обычно получается так:

- без dropout train accuracy выше;
- но gap между train и val больше;
- с dropout обучение может идти медленнее, зато val-кривая получается устойчивее.

8.6. Функция `plot_overfit_compare(...)`

Она строит две пары кривых:

- loss для train/val без регуляризации и с dropout;
- accuracy для train/val без регуляризации и с dropout.

Это один из самых наглядных фрагментов всей лабораторной, потому что здесь видно не просто число на выходе, а **динамика**, по которой легко понять, что именно делает регуляризация.

9. Как читать результаты обучения правильно

9.1. Нормальное обучение

Обычно выглядит так:

- `train_loss` снижается;
- `val_loss` тоже снижается;
- `train_acc` и `val_acc` растут;
- разрыв между ними небольшой.

Это означает, что модель учит закономерность, а не просто запоминает train.

9.2. Недообучение

Признаки:

- и `train_acc`, и `val_acc` низкие;
- loss плохо уменьшается;
- кривые быстро выходят на плохое плато.

Причины могут быть такими:

- слишком слабая архитектура;
- слишком маленький learning rate;
- слишком мало эпох;

- слишком агрессивная регуляризация.

9.3. Переобучение

Признаки:

- `train_acc` продолжает расти;
- `val_acc` перестаёт расти или падает;
- `train_loss` падает, а `val_loss` начинает ухудшаться;
- разрыв между `train` и `val` увеличивается.

Именно это демонстрирует второй блок на маленькой выборке.

9.4. Быстрая, но нестабильная оптимизация

Если `lr` слишком большой, графики могут выглядеть шумно:

- `accuarcy` скачет;
- `loss` резко колеблется;
- модель может то улучшаться, то ухудшаться между эпохами.

Это не всегда означает плохую архитектуру — иногда проблема именно в шаге обучения.

10. Почему эта реализация хороша с инженерной точки зрения

В этой лабораторной есть несколько особенно удачных архитектурных решений.

10.1. Разделение ответственности по файлам

Это делает код проще для чтения и отладки:

- данные отдельно;
- модель отдельно;
- конфиг отдельно;
- эксперименты отдельно.

10.2. Повторное использование функций

Например:

- одна и та же `ImageCNN` используется в обоих блоках;
- `train_model(...)` одинаково применяется и для архитектурного перебора, и для оптимизаторов, и для `overfitting-demo`;
- `make_optimizer(...)` централизует создание оптимизаторов.

Это хороший признак того, что код написан не как одноразовый notebook, а как небольшой, но уже структурированный проект.

10.3. Воспроизводимость

Использование SEED везде, где это важно, делает выводы по экспериментам более надёжными.

10.4. Корректный ML-пайплайн

Лабораторная не смешивает роли выборок:

- train для обучения;
- val для выбора модели;
- test для финальной оценки.

Это одно из самых важных методологических достоинств всей работы.

11. Какой путь сигнала проходит один объект в сети

Чтобы окончательно понять CNN, полезно мысленно провести один объект через весь pipeline.

Пусть на вход подаётся одно изображение цифры 28 times 28.

1. `data.py` загружает изображение, превращает его в тензор и при необходимости нормализует.
2. `DataLoader` собирает его в батч размера `BATCH_SIZE`.
3. `ImageCNN.forward(x)` передаёт батч через `self.features`.
4. Каждый `ConvBlockA` или `ConvBlockB` строит новые карты признаков.
5. После всех блоков `AdaptiveAvgPool2d((1,1))` сводит каждую карту признаков к одному числу.
6. `Flatten()` превращает результат в вектор признаков.
7. `Dropout` во время обучения случайно зануляет часть компонентов.
8. `Linear` выдаёт 10 логитов — по одному на класс.
9. `CrossEntropyLoss` сравнивает логиты с истинной меткой.
10. На training шаге градиент идёт обратно по всей сети и обновляет фильтры и веса.

Если держать в голове именно этот путь, код перестаёт выглядеть набором отдельных функций и начинает восприниматься как единый вычислительный конвейер.

12. Что стоит понимать особенно хорошо для защиты или отчёта

Если тебе нужно уверенно объяснить эту лабораторную, то вот минимальный набор идей, который надо уметь проговаривать своими словами.

1. **Почему CNN лучше MLP для изображений.** Потому что CNN использует локальные области и общие веса, сохраняя пространственную структуру.
2. **Зачем нужен pooling.** Он уменьшает размер представления, выделяет сильные локальные признаки и помогает устойчивости к малым сдвигам.
3. **Почему нужна validation выборка.** Она используется для выбора архитектуры без утечки информации с test.
4. **Что делает early stopping.** Останавливает обучение, когда val_acc перестаёт улучшаться, и возвращает лучшее состояние модели.
5. **Зачем финальное обучение на полном train.** После выбора лучшей архитектуры разумно обучить её на всех доступных обучающих данных.
6. **Как распознать переобучение по графикам.** По растущему разрыву между train и val и ухудшению validation-кривых при продолжающемся улучшении train.
7. **Что даёт dropout.** Ослабляет переобучение, заставляя модель строить более устойчивые представления.

13. Что можно улучшить в дальнейшем

Если смотреть на лабораторную уже как на основу для дальнейших экспериментов, можно развивать её так:

- добавить BatchNorm2d после свёрток;
- попробовать аугментации изображений;
- вынести логирование результатов в csv или json;
- добавить сохранение лучших весов на диск;
- расширить сетку архитектур;
- сравнить AdaptiveAvgPool с более классическим полносвязным хвостом;
- протестировать более сложные датасеты.

Это не требуется для текущей лабы, но помогает понять, куда развивается реальный CNN-пайплайн в более серьёзных задачах.

14. Краткий итог по лабораторной

Эта работа показывает полный базовый цикл исследования CNN:

- подготовка и нормализация данных;
- проектирование архитектуры через свёрточные блоки;
- обучение с CrossEntropyLoss и оптимизатором;
- контроль качества по validation;
- ранняя остановка;
- выбор лучшей конфигурации;
- честная финальная оценка на test;
- анализ влияния оптимизатора и регуляризации.

Если после чтения этого гайда ты понимаешь:

- как устроен один свёрточный блок;
- чем `block A` отличается от `block B`;
- как из `train/val/test` строится корректный эксперимент;
- почему `Dropout` уменьшает переобучение;
- как код из `lab4/` реализует все эти идеи,

то тема лабораторной уже разобрана на хорошем уровне.

15. Практический порядок чтения кода после этого гайда

Чтобы закрепить материал, полезно пройти по коду в таком порядке:

1. Прочитать `config.py` и сопоставить параметры с экспериментами.
2. Прочитать `data.py` и проследить путь данных от загрузки до `DataLoader`.
3. Прочитать `cnn.py`, особенно `ConvBlockA`, `ConvBlockB`, `ImageCNN`, `train_model`.
4. Прочитать `block1_main.py` и проследить жизненный цикл одной конфигурации в сетке.
5. Прочитать `block2_main.py` и по графикам сравнить оптимизаторы и переобучение.

Если сделать это именно в таком порядке, то код будет восприниматься не как набор отдельных файлов, а как логически цельная система экспериментов над CNN.